

Automated Mirror Distortion Measurement System

Software Design Document

ENGR 498A - Senior Design

Team 23058 Members

Kevin Garcia
Mina Yoo
Isaac Pimienta
Francisco Navarro
Rob Purcell

Mentor

Mike Nofziger

Sponsor

Mike Lee
Nitin Patel
Adam Humeres
Clif Gerke
Caterpillar Inc.

Rev: 1

Revision History

Date	Revision	Description
15 Oct-2023	0	Initial Release
30 Mar-2024	1	Major updates in all sections for updated software

Table of Contents

<i>1.0 Scope</i>	<i>4</i>
<i>2.0 Referenced Documents</i>	<i>5</i>
<i>3.0 Acronyms</i>	<i>6</i>
<i>4.0 CSCI Design Decisions</i>	<i>6</i>
<i>5.0 CSCI Architectural Design</i>	<i>8</i>
<i>6.0 CSCI Detailed Design</i>	<i>9</i>
<i>7.0 Requirements Traceability</i>	<i>10</i>
<i>8.0 Notes</i>	<i>10</i>

1.0 Scope

This Software Design Document (SDD) describes the interface, functions, and architecture of the Graphical User Interface (GUI) / Display Computer Software Configuration Item (CSCI) for the Automated Mirror Distortion Measurement System (AMDMS).

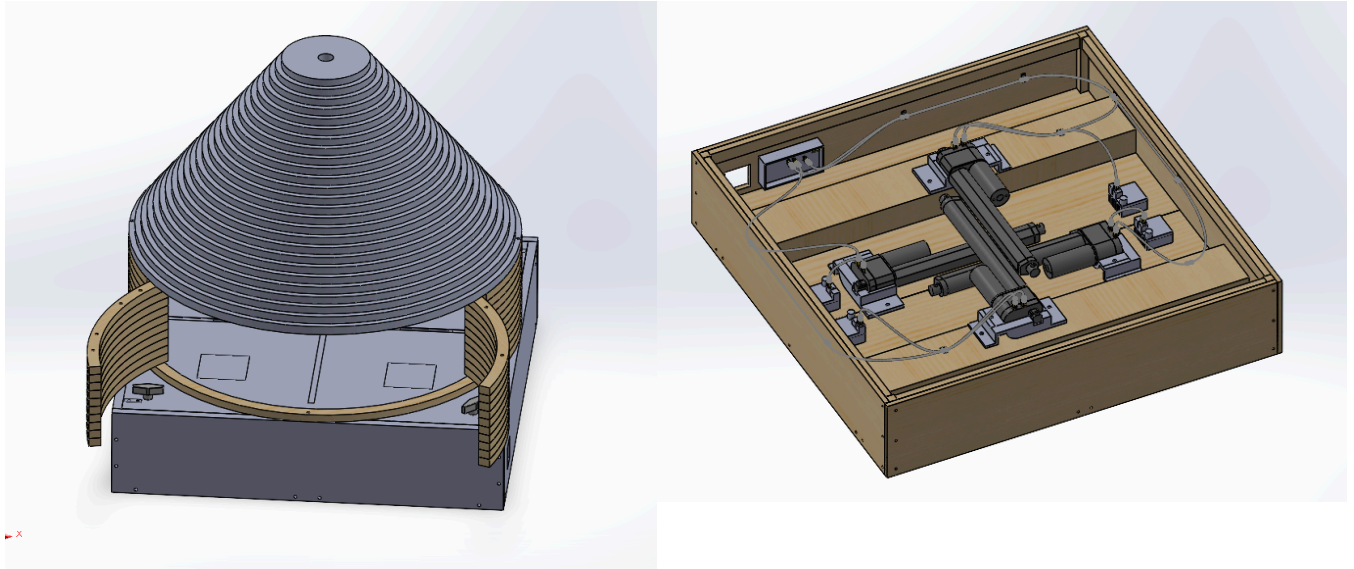


Figure 1: Autonomous Mirror Distortion Measurement System

The current MDMS can measure the distortion in convex mirrors and provide a PASS/FAIL output indicating the mirror distortion level within the Japanese Industrial Standard Mirrors for Automobiles (JIS5705) standard accepted range. The AMDMS will build from this by identifying the mirror inputted, storing the areas where distortion is found, providing real time analytics on mirrors measured in the past hour, and generating analytic reports for each kind of mirror.

2.0 Referenced Documents

3.0 Acronyms

SDD – Software Design Document

GUI – Graphical User Interface

CSCI – Computer Software Configuration Item

(A)MDMS – (Automated) Mirror Distortion Measurement System

CSC – Computer Software Component

ROI – Region of Interest

API – Application Programming Interface

FOV – Field of View

4.0 GUI / Display CSCI Design Decisions

Figure 3 shows the system block diagram. The core of the system is the CPU / GUI that hosts the GUI / Display CSCI software. As can be seen, the CPU / GUI controls all the functions of the AMDMS, including the hardware functions of calibrating the camera, centering the mirror, and grabbing images.

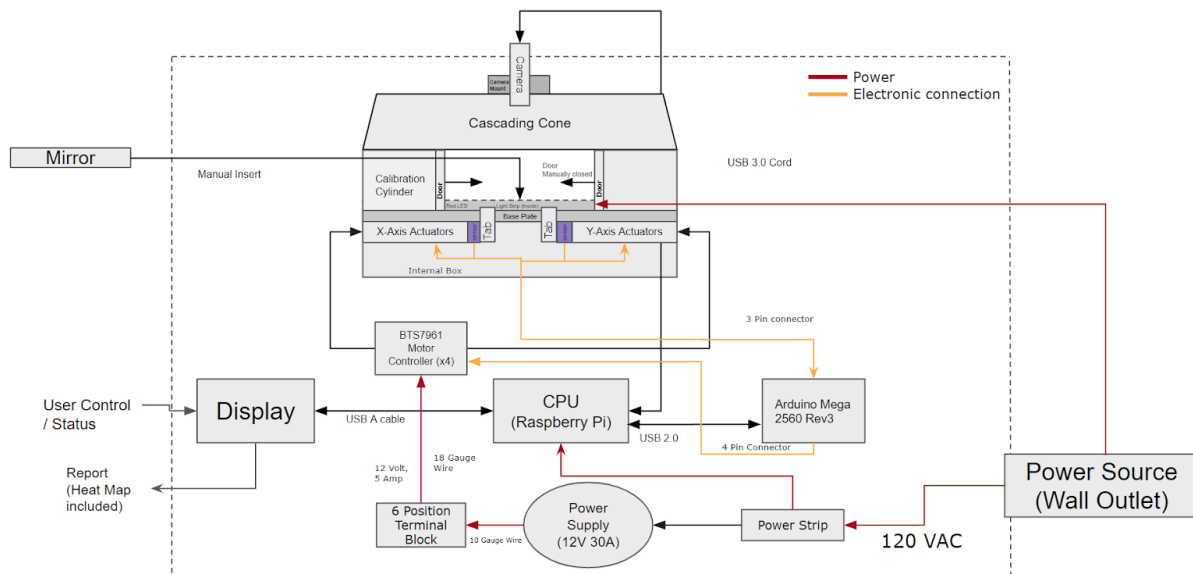


Figure 3: System Block Diagram

The GUI / Display CSCI also provides the user interface and software functions for each of its CSC modules. Because the main purpose of this system is automation, the user is only required to trigger functions by the touch of a single button. The user can call functions like: Calibrate Camera, Start Measurement, Change Theme, and Create Report. There are other functions that are called in between that don't require user input. It has functions of providing the last time the camera was calibrated, the last time a measurement was ran, a summary of the measurements performed in the last hour, and the results of the most recent measurement.

This software package contains the same image processing algorithm from the MDMS to measure the distortion of mirrors. Without the user's input, the algorithm measures a mirror's size, identifies the part number of the mirror, measures its distortion, and stores the results into a SQL database. In addition, the GUI has a function to create an analytics report based off the user's preference and delivers it to a specified email address.

GUI Design Layout

The following figures, 4 and 5, show the GUI layout in its light and dark themes. The GUI display includes a "Start Measurement" button, an "Emergency Stop" button, a "Calibrate

Camera” button, a “Generate Report” button. The GUI also real time analytics on the measurements that have been ran in the last hour per mirror type.

Mirror Distortion Measurement

Start measurement

Emergency Stop

CAT

Last Hour Summary

Mirror Part Number	Average Pass Rate	Total Passed	Total Failed	Average Distortion Percentage
221-9264 / 153-4010				
5P-6879				
8T-2287				

Most Recent Run

Part Num:

Results:

Distortion Percentage:

Last Run:

Last Calibration:

Generate Report

Calibrate Camera

Dark Mode

Figure 4: GUI Design

Mirror Distortion Measurement

Start measurement

Emergency Stop

CAT

Last Hour Summary

Mirror Part Number	Average Pass Rate	Total Passed	Total Failed	Average Distortion Percentage
221-9264 / 153-4010				
5P-6879				
8T-2287				

Most Recent Run

Part Num:	
Results:	
Distortion Percentage:	
Last Run:	
Last Calibration:	

Generate Report

Calibrate Camera

Light Mode

Figure 5: GUI Design

Mirror Distortion Measurement

The first of three main use cases is as follows; user clicks the “Start Measurement” button when they’ve loaded the mirror into the system. The GUI calls the Arduino Mega’s function to center the mirror to the center of the camera’s FOV. The alignment algorithm runs and centers the mirror. The software does not know where the center of the camera’s FOV is. We are achieving this result by aligning all the components in the system geometrically. After centering the mirror, the alignment algorithm in the Arduino identifies the size of the mirror according to the position of the actuators. The Arduino returns the values and the CPU the mirror size/part number and then runs the image processing algorithm to measure the distortion of the mirror. If the user needs to cancel the measurement at any point, they can press the “Emergency Stop” button to stop the measurement. The CPU stores the values in the SQL database and updates all the values in the GUI accordingly. The part numbers are predetermined by the sponsors through their requirements set for the system.

Figure 6 shows the SQL database and its required fields to store the values of every measurement that is ran. This database is used to generate analytics reports and update the values in the GUI. This database is stored in the local hard drive inside of the project folder. The size of each entry averages about 4kb in storage size which allows for up to 50,000 measurements to be stored in a 7 GB CPU. Due to the risk of maximizing the local storage, data will be purged after 90 days to ensure that storage is always available. The fields are as follows.

- id: a 16 character randomly generated identification
- coordinates: pixel coordinates of the areas where distortion was detected
- mirrorPartNum: the part number of the mirror associated to this given measurement
- distortionLevel: the distortion percentage calculated in the measurement
- runDate: date and time the measurement was ran
- result: Pass/Fail
- runTime: the time it took the complete the measurement

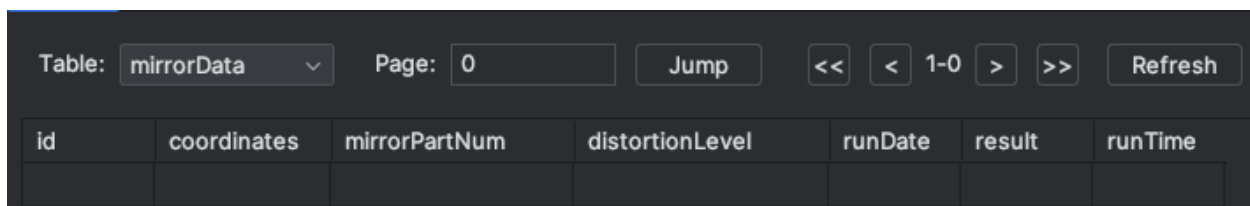


Table:	mirrorData	Page:	0	Jump	<<	<	1-0	>	>>	Refresh
id	coordinates	mirrorPartNum	distortionLevel	runDate	result	runTime				

Figure 6: Mirror Data Database

Report Generation

The second of three main use cases is as follows; user clicks the “Generate Report”. The GUI generates a new window that allows the user to select the preferences for their analytics report. Figure 7 shows a notional menu of the questions the user is required to answer.

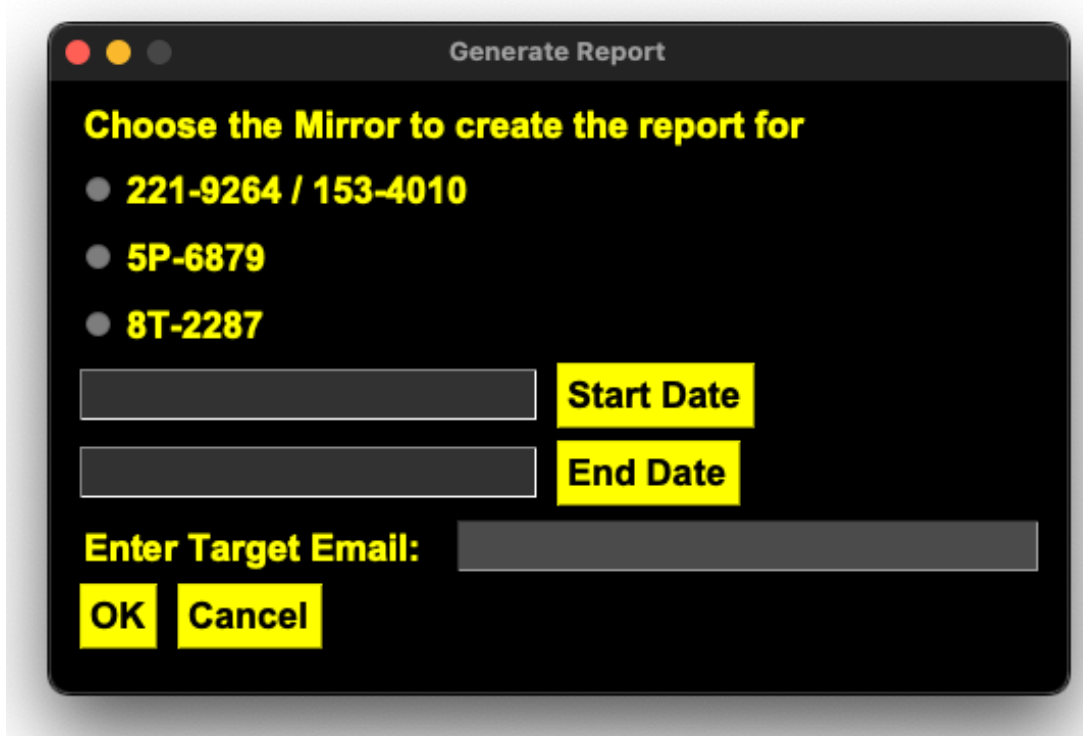


Figure 7: Generate Report Window

In addition, the software handles any exceptions in the case that there no inputted values or the values could cause any errors. The following figures show the software's error handling.

- A mirror part number is not selected for querying



Figure 8: Mirror Part Number Not Selected Exception Window

- A start date or end date is not selected

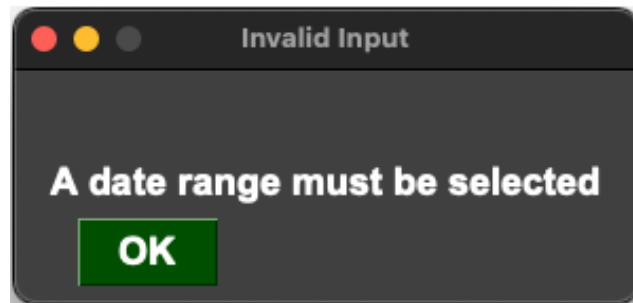


Figure 9: A Start Date or End Date is Not Selected Exception Window

- The start date is after the end date

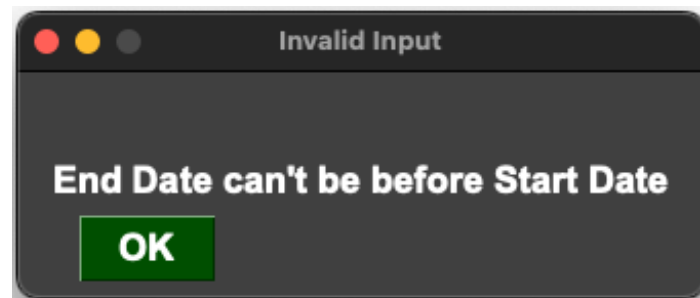


Figure 10: The End Date is Before the Start Date Exception Window

- The format of the inputted email does not match that of an email

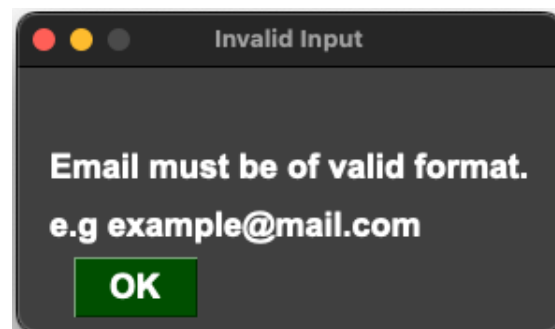


Figure 11: Inputted Email is Not Correct Format Exception Window

After the user has inputted their values, the software queries the database to get all the measurements that were ran within the given time frame (inclusive dates), for the chosen mirror part number. If it doesn't return any results, the Figure 12 is shown in a new window.

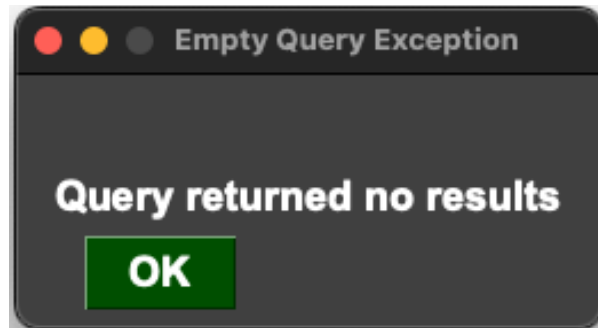


Figure 12: Empty Query Exception Window

Otherwise, the software package generates a PDF report showing some analytics of the measurements that were ran within the given time frame, for chosen mirror part number, and sent to the given email address. When the email is successfully sent, Figure 12 is displayed in a new window.

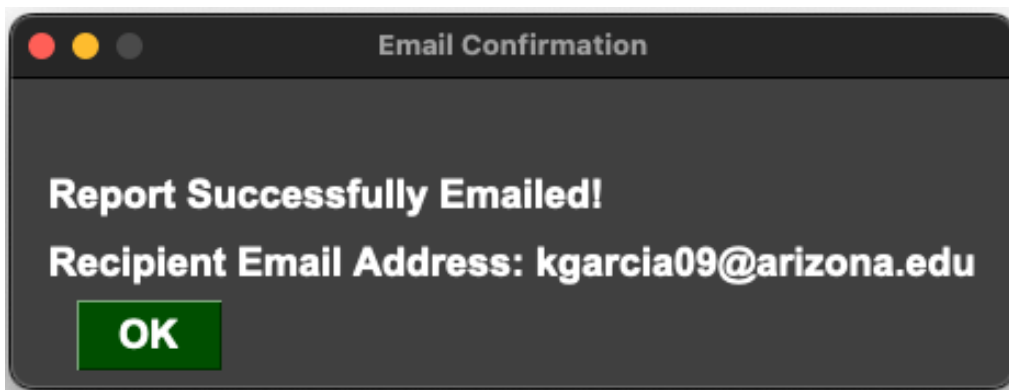


Figure 13: Email Confirmation Window

Figure 14 shows the email layout for the report delivery along with the attached report. Figure 15 shows the report with its generated analytics and a heatmap that shows the most common areas where distortion is found amongst the mirrors measured, with the given part number, within the given timeframe.

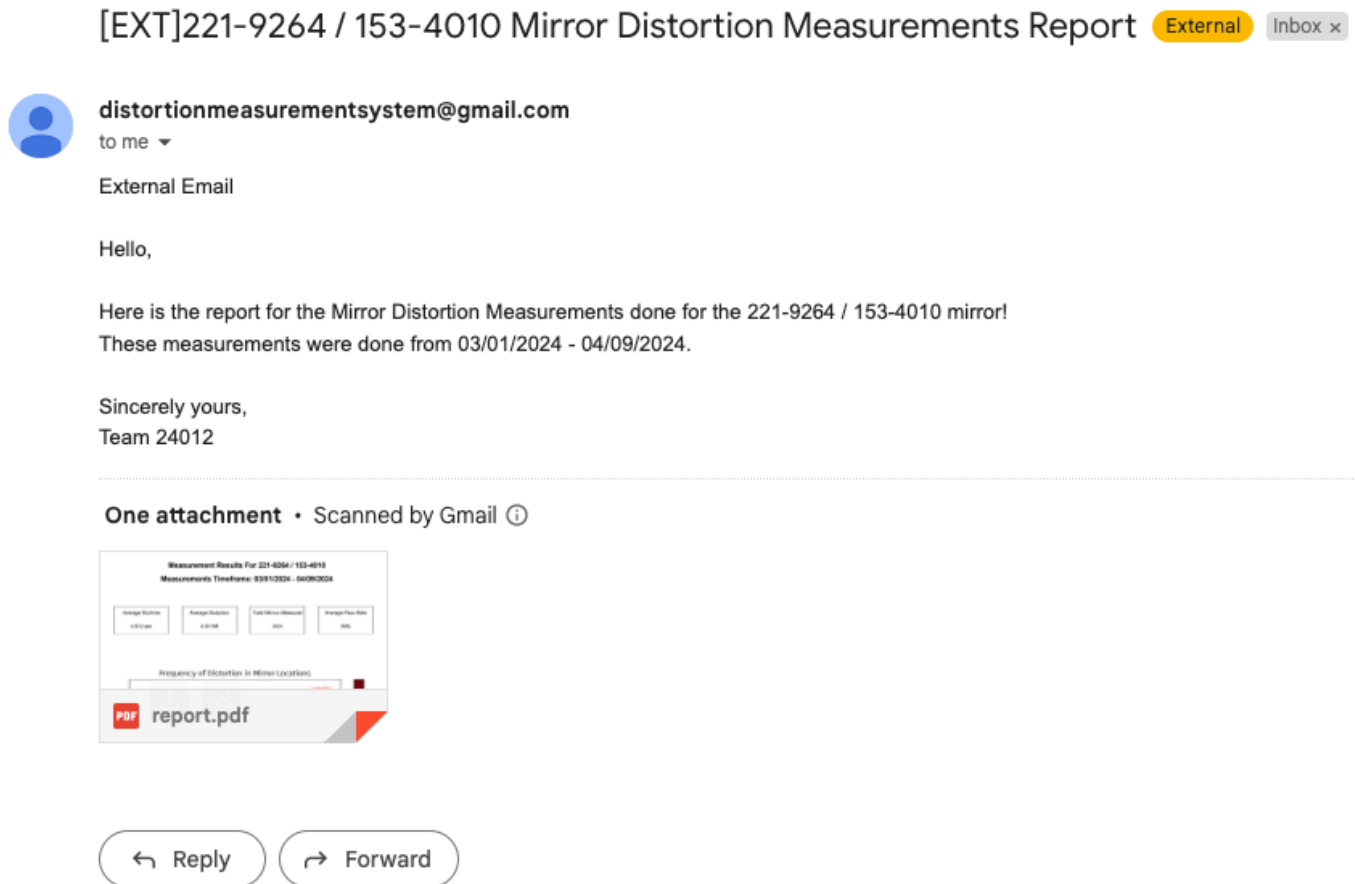


Figure 14: Email Delivery Layout

Measurement Results For 221-9264 / 153-4010
Measurements Timeframe: 03/01/2024 - 04/09/2024

Average Runtime	Average Distortion	Total Mirrors Measured	Average Pass Rate
0.512 sec	0.51155	3321	50%

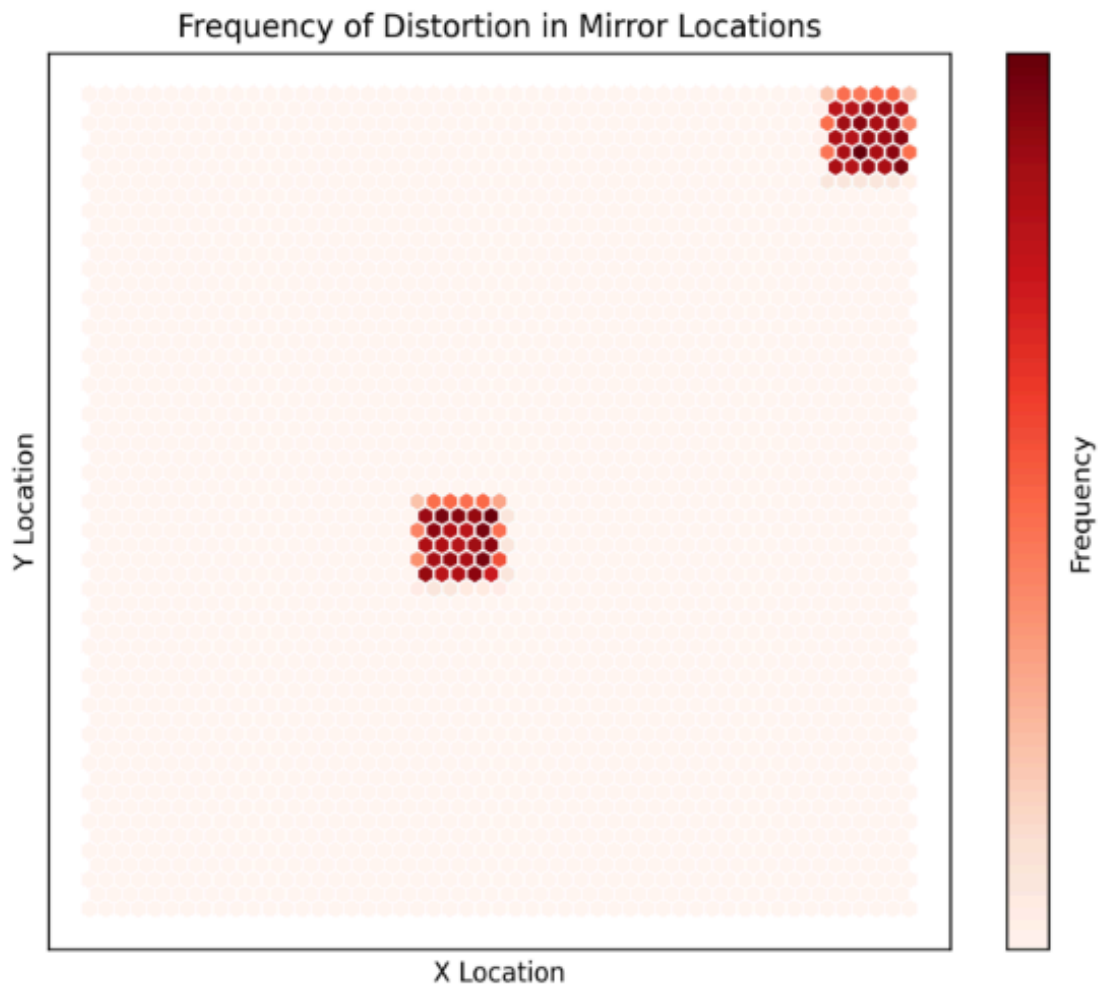


Figure 15: Generated Report PDF

5.0 CSCI Architectural Design

The algorithm consists of five computer software components which are all called by a computer software integration item on the GUI. The CSCs are a Camera API, Mirror Alignment CSC, Camera Interface CSC, Distortion Computation CSC, Data Storage and Run Time CSC, and Report Generation and Data Querying CSC.

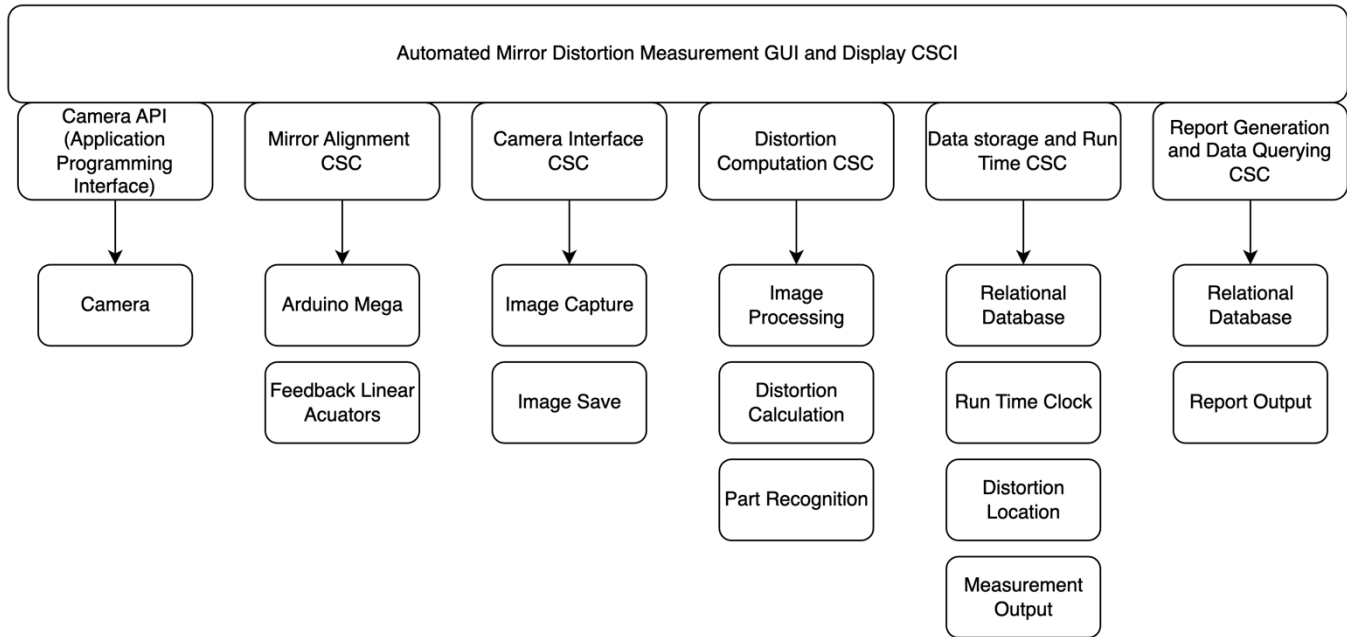


Figure 16: CSCI Architecture

6.0 CSCI Detailed Design

All the algorithms, data operations, and GUI displays will be programmed in Python. The Mirror Alignment CSC is programmed in C++ as it is an Arduino Embedded System. Most of the existing algorithms from the MDMS project stayed the same with some minor additions to account for data storage/querying.

Mirror Alignment CSC:

The mirror alignment CSC consists of a call from the Raspberry Pi (CPU) to the Arduino Mega, which runs a circuit to align the mirror. The circuit contains 4 linear actuators where every pair aligns the mirror in one axis. The entire circuit uses feedback loops to constantly track the location of the actuators and button sensors to track when the actuators are in contact with the mirror.

Camera Interface CSC:

The camera interface CSC is a series of short algorithms that will control the camera. The CSC will turn the camera on/off, send signals to take a picture and adjust the ROI. The existing algorithm effectively controls the camera and takes in clear pictures. Using OpenCV python

library, the algorithm can identify the cone's reflection through the mirror and find the edges of the mirror given its predetermined size. The main addition for this algorithm is to add dynamic adjustments for the ROI. The algorithm shall dynamically identify the edges of the mirror.

Distortion Computation CSC:

The distortion computation component is a series of python functions that calculate the distortion in the input image. The existing algorithm effectively measures the distortion of the image and requires no adjustment.

Data Storage and Run Time CSC:

This CSC will be an addition to the Distortion Computation CSC. As the computation is happening, there will be some storage of the pixel coordinates being measured as well as the distortion level. These values will be stored in a SQLite3 database, which a python library that creates a relational database in the project's directory. The database will have columns for part_number, pass/fail, distorted_area, distortion_level, and runtime. The Run Time component is already implemented.

Report Generation and Data Querying CSC:

This algorithm will only run when the user interfaces with the GUI and requests a report. The report can contain measurement results for any part number during a specified period. The CSC will be a simple algorithm that'll take in the user's input and use it to query data from the SQLite3 database.

Data Querying Algorithm:

```
Part_num_requested = input from gui
Period_requested = input from gui
con.close()
new_con = sqlite3.connect("mirrorData.db")
new_cur = new_con.cursor()
query = new_cur.execute(query using input values)
```

Using the query results from the code above, a matplotlib histogram will be created using the two arrays of distorted area and distortion level.

7.0 Requirements Traceability

The software of the AMDMS has the following requirements:

- The software shall be implemented in Python.
- The algorithm shall take less than or equal to 30 seconds.
- The software shall generate reports according to part number measurements.

8.0 Notes

The software package will be delivered in a zip file to the sponsors.

The GitHub repository: <https://github.com/garciak09/AMDMS-SoftwarePackage>

The project folder contains all the code necessary to run the AMDMS as well as comments in every function to explain the logic as well as the functionality.

To satisfy dependencies, install the following:

- fpdf
- matplotlib
- numpy
- Sqlite3
- smtplib
- opencv_python
- opencv_python_headless
- pandas
- pdfkit
- Pillow
- pypylon
- PySimpleGUI
- rembg
- scipy
- ydata_profiling
- Tkinter

The software package was run on a Raspberry Pi 4 Computer Model B and a Macbook Air with an M1 chip.

The old GUI is still in this project folder. The new software package with all its modifications and additions are only connect to newGUI.py.

testAPI.py is a file used to test the database CRUD (Create, Read, Update, Deleted) operations.

PySimpleGUI is a great tool for displaying a quick GUI and calling supporting functions in the backend. However, as more features are added PySimpleGUI's code structure can get a little messy due to its limited functionality.